



Отчет по лабораторной работе

Задание 1 по курсу «Информационные технологии» вариант 11

Учебная группа

М7О-207БВ-24

Студент

Никитин П.А. _____

Преподаватель

Барчев Н.Б. _____

Содержание

1 Текст задания лабораторной работы	1
2 Псевдокод	2
3 Сведения о программной реализации	5
3.1 Информация об используемых языках программирования и их версиях, о версиях систем программирования	5
3.2 Описание входных и выходных данных	5
3.3 Краткое описание спроектированных программных единиц	6
3.4 Список диагностических сообщений с указанием возможных причин их появления	9
4 Инструкция использования программы	10
5 Листинг программной разработки	12
6 Результаты тестирования	19

1

Текст задания лабораторной работы

Задание 1 по курсу «Информационные технологии» вариант 11

1. Подготовить программу, формирующую на основе информации, вводимой пользователем с клавиатуры, два внешних текстовых файла:
 - файл автомобилей: состоит из записей, каждая из которых включает три поля – марки, модели и номерного знака;
 - файл регистрационных сведений: состоит из записей, каждая из которых включает четыре поля – номерного знака, фамилии и адреса владельца, года выпуска.
2. В процессе проектирования предусмотреть необходимые по смыслу задания проверки корректности данных, а также адекватное задаче взаимодействие с пользователем. **Учесть, что ограничения на размеры файлов отсутствуют, файлы в общем случае могут иметь различные длину.**
В обязательном порядке использовать языковые средства организации программных единиц.
Не использовать программные средства, поддерживающие работу с базами данных.
3. Программу подготовить с использованием средств одной из реализаций языка программирования C++, **ввод и вывод информации реализовать при помощи средств языка программирования С.** Выполнить тестирование обеих программ согласно стратегии черного ящика, используя соответствующие критерии тестирования.

2

Псевдокод

```
1 файл main.cpp:
2
3 ПОДКЛЮЧЕНИЕ СОДЕРЖИМОГО "file_handler.hpp"
4 ВЫЗОВ функции main()
5
6 функция main():
7     УСТАНОВИТЬ ЛОКАЛЬ: en_RU.UTF-8
8     ВЫВЕСТИ: Назначение программы
9     ВЫВЕСТИ: Возможные опции
10    ПОКА ПОЛЬЗОВАТЕЛЬ НЕ ВВЕЛ EOF:
11        ОБРАБОТАТЬ ПОТОК ВВОДА ДЛЯ ВЗАИМОДЕЙСТВИЯ С ПРОГРАММОЙ
12        ЕСЛИ ПОЛЬЗОВАТЕЛЬ ЗАВЕРШИЛ РАБОТУ С ПРОГРАММОЙ ТО:
13            ВЫЙТИ ИЗ ЦИКЛА
14        ОЖИДАТЬ ПОЛЬЗОВАТЕЛЬСКИЙ ВВОД
15    ВЫВЕСТИ: Сообщение о том, что программа завершена.
16    ЗАВЕРШИТЬ ПРОГРАММУ
17    ВЕРНУТЬ 0
18
19 файл file_handler.cpp:
20
21 ПОДКЛЮЧЕНИЕ СОДЕРЖИМОГО "file_handler.hpp"
22
23 ПРОИНИЦИАЛИЗИРОВАТЬ глобальные переменные: СТАНДАРТНОЕ_ИМЯ_ФАЙЛА_1,
    СТАНДАРТНОЕ_ИМЯ_ФАЙЛА_2, НИЖНЯЯ_ГРАНИЦА
24
25 ОБЪЯВИТЬ БУФФЕР[256] // ИЗ 256 ЭЛЕМЕНТОВ
26
27 функция clear_stdin():
28    ОЧИЩАЕТ ПОТОК ВВОДА
29
30 функция validate_filename(имя_файла):
31    ПРОИНИЦИАЛИЗИРОВАТЬ регулярное_выражение_файла{"[a-zA-Z0-9_-]+\.\.txt"}
32    ВЕРНУТЬ имя_файла СООТВЕТСТВУЕТ регулярное_выражение_файла
33
34 функция validate_license(номерной_знак):
35    ПРОИНИЦИАЛИЗИРОВАТЬ регулярное_выражение_номерного_знака{"[АВЕКМНОРСТУХ][0-9]{3,3}
    [АВЕКМНОРСТУХ]{2,2}[0-9]{2,3}RUS"}
36    ВЕРНУТЬ номерной_знак СООТВЕТСТВУЕТ регулярное_выражение_номерного_знака
37
38 функция validate_brand(бренд):
39    ПРОИНИЦИАЛИЗИРОВАТЬ регулярное_выражение_бренда{"[А-ЯЁа-яёА-Za-z0-9-_.! ]+"}
40    ВЕРНУТЬ бренд СООТВЕТСТВУЕТ регулярное_выражение_бренда
```

```

41
42 функция validate_model(модель):
43     ПРОИНИЦИАЛИЗИРОВАТЬ регулярное_выражение_модели{"[А-ЯЁа-яёА-Za-z0-9-_.!() ]+"}
44     ВЕРНУТЬ модель СООТВЕТСТВУЕТ регулярное_выражение_модели
45
46 функция validate_surname(фамилия):
47     ПРОИНИЦИАЛИЗИРОВАТЬ регулярное_выражение_фамилии{"[А-ЯЁа-яёА-Za-z]+"}
48     ВЕРНУТЬ фамилия СООТВЕТСТВУЕТ регулярное_выражение_фамилии
49
50 функция validate_address(адрес):
51     ПРОИНИЦИАЛИЗИРОВАТЬ регулярное_выражение_адреса{"[А-Za-zА-ЯЁа-яё0-9!_- . ]"}
52     ВЕРНУТЬ адрес СООТВЕТСТВУЕТ регулярное_выражение_адреса
53
54 функция validate_release_year(год_выпуска):
55     ПРОИНИЦИАЛИЗИРОВАТЬ результат = ИСТИНА
56     ЕСЛИ год_выпуска ПУСТОЙ ИЛИ СОДЕРЖИТ НЕЦИФРЫ ТО:
57         ВЫВЕСТИ: Сообщение о неправильном вводе.
58         ВЕРНУТЬ ЛОЖЬ
59
60     ПРОИНИЦИАЛИЗИРОВАТЬ текущий_год = ТЕКУЩИЙ ГОД СИСТЕМЫ
61     ЕСЛИ год_выпуска < НИЖНЯЯ_ГРАНИЦА ИЛИ год_выпуска > ТЕКУЩИЙ ГОД СИСТЕМЫ ТО:
62         ВЫВЕСТИ: "Год вне диапазона"
63         результат = ЛОЖЬ
64     ВЕРНУТЬ результат
65
66 функция file_info(стандартное_имя_файла):
67     ПРИНЯТЬ пользовательский ввод
68     ОБРАБОТАТЬ ПОТОК ВВОДА И ОПРЕДЕЛИТЬ имя_файла
69     ОБРАБОТАТЬ ПОТОК ВВОДА И ОПРЕДЕЛИТЬ режим_открытия_файла
70     ВЕРНУТЬ имя_файла И режим_открытия_файла
71 функция write_car_data():
72     ПРОИНИЦИАЛИЗИРОВАТЬ имя_файла, режим_открытия_файла =
file_info(СТАНДАРТНОЕ_ИМЯ_ФАЙЛА_1)
73     ПРОИНИЦИАЛИЗИРОВАТЬ файл(имя_файла) В РЕЖИМЕ ОТКРЫТИЯ режим_открытия_файла
74
75     ЕСЛИ файл НЕ ОТКРЫВАЕТСЯ ТО:
76         ВЫВЕСТИ: "Не удалось открыть файл %имя_файла\n"
77         ВЕРНУТЬ
78     ВЫВЕСТИ: "Файл %имя_файла успешно открыт для записи\n"
79
80     ПОКА ПОЛЬЗОВАТЕЛЬ НЕ ЗАВЕРШИЛ ВВОД ДАННЫХ:
81         ЕСЛИ ПОЛЕ ВВЕДЕНО НЕПРАВИЛЬНО ТО:
82             ВЫВЕСТИ: Сообщение об ошибке в введенном поле
83             ПОВТОРИТЬ ВВОД ПОЛЯ
84         ЕСЛИ ВСЕ ПОЛЯ ВВЕДЕНЫ ПРАВИЛЬНО:
85             ЗАПИСАТЬ ДАННЫЕ В файл
86             ВЫВЕСТИ: Сообщение о введенной записи
87         ЕСЛИ ПОЛЬЗОВАТЕЛЬ ЗАВЕРШИЛ ВВОД ДАННЫХ ТО:
88             ЗАВЕРШИТЬ ЦИКЛ
89
90     ЗАКРЫТЬ файл
91     ОЧИСТИТЬ ПОТОК ВВОДА
92     ВЕРНУТЬ
93
94 функция write_license_data():
95     ПРОИНИЦИАЛИЗИРОВАТЬ имя_файла, режим_открытия_файла =
file_info(СТАНДАРТНОЕ_ИМЯ_ФАЙЛА_2)
96     ПРОИНИЦИАЛИЗИРОВАТЬ файл(имя_файла) В РЕЖИМЕ ОТКРЫТИЯ режим_открытия_файла
97
98     ЕСЛИ файл НЕ ОТКРЫВАЕТСЯ ТО:
99         ВЫВЕСТИ: "Не удалось открыть файл %имя_файла\n"
100     ВЕРНУТЬ
101     ВЫВЕСТИ: "Файл %имя_файла успешно открыт для записи\n"

```

```
102
103 ПОКА ПОЛЬЗОВАТЕЛЬ НЕ ЗАВЕРШИЛ ВВОД ДАННЫХ:
104     ЕСЛИ ПОЛЕ ВВЕДЕНО НЕПРАВИЛЬНО ТО:
105         ВЫВЕСТИ: Сообщение об ошибке в введенном поле
106         ПОВТОРИТЬ ВВОД ПОЛЯ
107     ЕСЛИ ВСЕ ПОЛЯ ВВЕДЕНЫ ПРАВИЛЬНО:
108         ЗАПИСАТЬ ДАННЫЕ В файл
109         ВЫВЕСТИ: Сообщение о введенной записи
110     ЕСЛИ ПОЛЬЗОВАТЕЛЬ ЗАВЕРШИЛ ВВОД ДАННЫХ ТО:
111         ЗАВЕРШИТЬ ЦИКЛ
112
113 ЗАКРЫТЬ файл
114 ОЧИСТИТЬ ПОТОК ВВОДА
115 ВЕРНУТЬ
```

3

Сведения о программной реализации

3.1 Информация об используемых языках программирования и их версиях, о версиях систем программирования

Языки программирования: C++17 (GNU C Compiler 15.2.1)

Утилита сборки программы: GNU Make 4.4.1

Системы программирования: Visual Studio Code 1.106.0

3.2 Описание входных и выходных данных

Входные данные:

Пользователь выбирает опции функционирования программы и записывает данные в выбранный файл посредством ввода текста. Ввод представляет собой номер выбранной опции – число, имя файла – текст, с которым будет связано дальнейшее выполнение программы, а также последовательный ввод полей для записи в файл – текст

Структура входных данных:

данные вводятся по одному на строку, таким образом пользователь может ввести только одну опцию за раз, а заполнение полей записей файла происходит по одному на строку (одно поле – одна строка)

Выходные данные и их структура:

Пользователь имеет два вида выходных данных:

1. Консольные сообщения: представляют собой текст, необходимый пользователю для пользования программой. Сообщения включают в себя: назначение программы, опции пользования программой, информацию об успешном/неуспешном открытии файла, предупреждения, если пользователь неправильно ввел поле записи, а также сообщение о завершении программы.
2. Файлы: представляют из себя текстовые файлы, содержащие записи, введенные пользователем в порядке записи по одному на строку, имя файла либо вводится пользователем, либо присваивается стандартное программой: `car_data.txt` или `license_data.txt`, в зависимости от назначения программы.

Файл автомобильных сведений:

состоит из записей, каждая из которых содержит три поля – марка, модель и номерной знак.

Файл регистрационных сведений:

состоит из записей, каждая из которых содержит четыре поля – номерной знак, фамилия, адрес владельца и год выпуска.

3.3 Краткое описание спроектированных программных единиц

файл main.cpp:

Функция	Назначение	Аргументы	Возвращает
main	Точка входа программы	Отсутствуют	Код завершения программы. Тип: int

файл file_handler.hpp:

Подключает следующие библиотеки:

Библиотека	Назначение
cstdio	библиотека стандартного ввода/вывода C
string	библиотека для работы с «умными» строками
regex	библиотека для работы с регулярными выражениями
cstdlib	стандартная библиотека C
filesystem	библиотека для работы с файловой системой, нужна для проверки существования файла

Содержит следующие структуры:

Структура	Назначение	Поля
CarData	Структура для хранения автомобильных сведений	license – поле хранит информацию про номерной знак автомобиля, brand – поле хранит информацию про бренд автомобиля, model – поле хранит информацию про модель автомобиля
LicenseData	Структура для хранения регистрационных сведений	license – поле хранит информацию про номерной знак автомобиля, surname – поле хранит информацию про фамилию владельца автомобиля, address – поле хранит информацию про адрес владельца автомобиля, release_year – поле хранит год выпуска автомобиля

Также содержит все прототипы функций: (подробнее о функциях ниже)

Прототип функции
<code>void clear_stdin()</code>
<code>bool validate_filename(std::string filename)</code>
<code>bool validate_license(std::string license)</code>
<code>bool validate_brand(std::string brand)</code>
<code>bool validate_model(std::string model)</code>
<code>bool validate_surname(std::string surname)</code>
<code>bool validate_address(std::string address)</code>
<code>bool validate_release_year(std::string release_year)</code>
<code>std::pair<std::string, std::string> file_info(std::string default_filename)</code>
<code>void write_car_data()</code>
<code>void write_license_data()</code>

файл `file_handler.cpp`:

Подключает содержимое `file_handler.hpp` и содержит реализации функций

Под правильностью введенных полей, имен файлов, понимается:

1. в случае полей: это отсутствие *неподходящих символов* и *правильный* порядок элементов.
2. в случае имени файла: это *подходящее* имя файла, которое операционная система способна создать.

Функция	Назначение	Аргументы	Возвращает
<code>clear_stdin</code>	Очистка потока ввода	Отсутствуют	Ничего. Тип: void
<code>validate_filename</code>	Валидация имени файла	filename – имя файла. Тип: std::string	true если имя верно, false если неверно. Тип: bool
<code>validate_license</code>	Валидация поля номерной знак	license – содержимое поля номерной знак. Тип: std::string	true если номерной знак введен верно, иначе false. Тип: bool
<code>validate_brand</code>	Валидация поля бренд	brand – содержимое поля бренд. Тип: std::string	true если бренд введен верно, иначе false. Тип: bool
<code>validate_model</code>	Валидация поля модель	model – содержимое поля модель. Тип: std::string	true если модель введена верно, иначе false. Тип: bool
<code>validate_surname</code>	Валидация поля фамилия	surname – содержимое поля фамилия. Тип: std::string	true, если фамилия введена верно, иначе false. Тип: bool

Функция	Назначение	Аргументы	Возвращает
validate_address	Валидация поля адрес	address – содержимое поля адрес. Тип: std::string	true, если адрес введен верно, иначе false. Тип: bool
validate_release_year	Валидация поля год выпуска	release_year – содержимое поля год выпуска. Тип: std::string	true если год выпуска введен верно и подходит по временному интервалу, иначе false. Тип: bool
file_info	Спрашивает у пользователя имя файла и режим работы с ним, для последующей работы с файлом	default_file_name – стандартное имя файла, в зависимости от выбранной опции, принимает соответствующее стандартное имя файла. Тип: std::string	пару значений, первое – это имя файла, второе – это режим открытия файла. Тип: std::pair<std::string, std::string>
write_car_data	Запись введенных пользователем сведений в файл автомобильных сведений, учитывая проверки на корректность ввода	Отсутствуют	Ничего Тип: void
write_license_data	Запись введенных пользователем сведений в файл регистрационных сведений, учитывая проверки на корректность ввода	Отсутствуют	Ничего Тип: void

3.4 Список диагностических сообщений с указанием возможных причин их появления

Сообщение	Возможная причина
Введен %с программа продолжает работу\n	Введенный символ не является N,n,Q,q или «enter»(«\n»).
Введен %с. Можно ввести только опцию 1 или опцию 2. Попробуйте еще раз.«	Введенный символ не является 1 или 2.
Введена пустая строка	Пользователь ввел enter(«\n») или что-то с нулевой длиной в поле год выпуска.
Строка не полностью состоит из цифр«	Поле год, введенное пользователем, содержит что-то кроме цифр.
Год вне диапазона(%d-%d)«	Поле год, введенное пользователем, выходит за описанные временные рамки.
Введенное имя файла некорректно. Файл должен иметь вид file.txt, где file состоит из латинских символов, дефиса(-) или нижнего подчеркивания(_), а после следует его расширение .txt«	Пользователь ввел неподходящее имя файла.
Файл %s уже существует.\nЖелаете продолжить запись вместо заполнения с начала(enter=Да/N=Нет)?	Пользователь ввел имя файла, который уже существует.
Не удалось открыть файл %s\n	Файл может не открываться, если неправильно введено имя файла, или с ним уже происходит работа.
Поле введено некорректно, попробуйте еще раз\n	Поле не соответствует заданному примеру/форме или заданному множеству элементов.

4

Инструкция использования программы

Инструкция полностью соответствует указаниям на экране.

Но все же рассмотрим как пользоваться программой:

Для запуска программы пользователь должен запустить файл: `./file_h`

Затем пользователь увидит следующее:

Назначение программы: Создание файлов на основе сведений, вводимых пользователем.

Что требуется записать?

- 1) Файл автомобильных сведений
- 2) Файл регистрационных сведений
- q) Для завершения программы нажмите q.

Теперь у пользователя есть выбор, как взаимодействовать с программой.

Рассмотрим основные варианты:

Допустим пользователь ввел 1. Теперь он увидит следующее:

Введите название файла(enter=car_data.txt):

Программа ожидает от пользователя название файла, в который будут записаны сведения. Если пользователь введет enter, то он будет работать с файлом `car_data.txt`. Если пользователь введет любое другое имя файла, которое удовлетворяет требованиям, например `file1.txt`, он увидит следующие сообщения:

Файл `file1.txt` успешно открыт для записи.

Начинаем заполнять автомобильные сведения.

Для завершения ввода нажмите 'Q' или 'q'.

Введите поле бренд (символы: А-ЯЁа-яА-Яа-я0-9-_.!):

Но если файл `file1.txt` уже существует будет выведено следующее сообщение:

Файл file1.txt уже существует.
Желаете продолжить запись вместо заполнения с начала(enter=Да/N=Нет)?

В таком случае программа в зависимость от ввода пользователя будет либо дописывать сведения в файл, либо заполнит его с нуля.

Теперь программа ожидает от пользователя заполнение поля бренд.

Если пользователь заполнит его без ошибок, то он перейдет к записи следующего поля. Если же пользователь совершит ошибку при вводе, то он увидит следующее сообщение:

Поле введено некорректно, попробуйте еще раз.
Введите поле бренд (символы: А-ЯЁа-яА-Яа-я0-9-_.!):

Это сообщение будет повторяться до тех пор, пока пользователь не введет поле правильно, или не решит завершить ввод, нажав q или Q. Как только пользователь правильно введет поле, он перейдет к заполнению следующего и так далее, пока не завершит ввод сведений, нажатием клавиши q или Q.

Стоит отметить, что данные записываются в файл при условии, что все каждое из полей было введено верно и при этом, все поля были заполнены.

Как только пользователь заполнит первую запись он увидит сообщение:

Запись 1 завершена:
бренд: Toyota
модель: Supra
номер: M976MM777RUS
Продолжаем ввод сведений.
Введите поле бренд (символы: А-ЯЁа-яА-Яа-я0-9-_.!):

Если пользователь введет q или Q он увидит следующее сообщение:

Желаете продолжить запись? (enter=Да/N=нет)

Теперь программа ожидает от пользователя выбор либо продолжить запись, завершить работу с программой. Если пользователь введет enter, он снова увидит сообщение о выборе опций взаимодействия с программой.

А если пользователь решит завершить работу с программой он увидит данное сообщение:

Нажмите любую клавишу для завершения программы...

Затем после нажатия любой клавиши, будет выведено следующее сообщение:

Программа завершена.

5

Листинг программной разработки

файл main.cpp:

```
1 #include "file_handler.hpp"
2
3
4 // точка входа программы
5 int main() {
6     // устанавливаем локаль UTF-8 с поддержкой кириллицы
7     setlocale(LC_ALL, "en_RU.UTF-8");
8     // Выводим назначение программы
9     printf("Назначение программы: Создание файлов на основе сведений, вводимых
пользователем\n");
10    // Начинаем взаимодействие с пользователем.
11    printf("Что требуется записать?\n1) Файл автомобильных сведений\n2) Файл
регистрационных сведений\n");
12    printf("q) Для завершения программы нажмите q.\n");
13    char input;
14    // Считываем и обрабатываем пользовательский ввод
15    while ((input = getchar()) != EOF) {
16        clear_stdin();
17        if (input == '1' || input == '2') {
18            if (input == '1') write_car_data();
19            else
20                if (input == '2') write_license_data();
21        } else if (input == 'q' || input == 'Q') {
22            break;
23        }
24        else printf("Данной опции не существует.\n");
25
26        printf("Желаете продолжить запись? (enter=Да/N=Нет)\n");
27        input = getchar();
28        if (input == 'N' || input == 'n') {
29            break;
30        } else
31            if (input == '\n') {
32                printf("Введен enter, программа продолжает работу\n");
33            } else printf("Введен %c, программа продолжает работу\n", input);
34
35        printf("Что требуется записать?\n1) Файл автомобильных сведений\n2) Файл
регистрационных сведений\n");
36        printf("q) Для завершения программы нажмите q.\n");
```

```

37     }
38
39     // Сообщаем пользователю о завершении программы и
40     // ожидаем нажатие любой клавиши
41     printf("Нажмите любую клавишу для завершения программы...");
42     scanf("%c", &input);
43     printf("Программа завершена.\n");
44     // Возвращаем ОС код успешного выполнения программы.
45     return 0;
46 }

```

файл file_handler.hpp:

```

1  #ifndef FILE_HANDLER_HPP
2  #define FILE_HANDLER_HPP
3
4  #include <cstdio> // библиотека стандартного С ввода/вывода
5  #include <string> // библиотека для работы с "умными" строками
6  #include <regex> // библиотека для работы с регулярными выражениями
7  #include <cstdlib> // стандартная библиотека С
8  #include <filesystem> // библиотека для работы с файловой системой (требуется C++17)
9
10 // Структура для временного хранения автомобильных сведений
11 struct CarData {
12     std::string license;
13     std::string brand;
14     std::string model;
15 };
16
17 // Структура для временного хранения регистрационных сведений
18 struct LicenseData {
19     std::string license;
20     std::string surname;
21     std::string address;
22     std::string release_year;
23 };
24
25 // Ниже указаны прототипы функций,
26 // подробнее о них в файле реализаций file_handler.cpp
27
28 // прототип функции очистки ввода(костыль)
29 void clear_stdin();
30
31 // прототипы функций валидации
32 bool validate_filename(std::string filename);
33 bool validate_license(std::string license);
34 bool validate_brand(std::string brand);
35 bool validate_model(std::string model);
36 bool validate_surname(std::string surname);
37 bool validate_address(std::string address);
38 bool validate_release_year(std::string release_year);
39
40 // прототипы функций, связанных с работой с файлами
41 std::pair<std::string, std::string> file_info(std::string default_filename);
42 void write_car_data();
43 void write_license_data();
44
45 #endif

```

файл file_handler.cpp

```

1  #include "file_handler.hpp"

```

```

2
3 // основные стандартные параметры
4 std::string DEFAULT_FILENAME_1 = "car_data.txt";
5 std::string DEFAULT_FILENAME_2 = "license_data.txt";
6 const int LOWER_BOUND_YEAR = 1960;
7
8 // буффер ввода
9 char BUFFER[256];
10
11 // функция очистки потока ввода,
12 // fseek() не подходит, тк не работает под Linux
13 void clear_stdin() {
14     int temp;
15     while ((temp = getchar()) != EOF && temp != '\n');
16 }
17
18 // Стоит отметить, что почти все функции валидации используют
19 // регулярные выражения, сделано это для упрощения проверок
20
21
22 // функция валидации имени файла
23 bool validate_filename(std::string filename) {
24     std::regex regex_filename{"[a-zA-Z0-9_-]+\\.txt"};
25     return std::regex_match(filename, regex_filename);
26 }
27
28 // функция валидации поля "номерной знак"
29 bool validate_license(std::string license) {
30     // M976MM777RUS
31     std::regex pattern{"[ABEKMHOPTX][0-9]{3,3}[ABEKMHOPTX]{2,2}[0-9]{2,3}RUS"};
32     return regex_match(license, pattern);
33 }
34
35 // функция валидации поля "бренд"
36 bool validate_brand(std::string brand) {
37     std::regex pattern{"[ФфРрТтУуХхЦцЧчШшЩщЪъЬьЭэЮюЁёЫыА-Яа-яА-Za-z0-9-_.! ]+"};
38     return regex_match(brand, pattern);
39 }
40
41 // функция валидации поля "модель"
42 bool validate_model(std::string model) {
43     std::regex pattern{"[ФфРрТтУуХхЦцЧчШшЩщЪъЬьЭэЮюЁёЫыА-Яа-яА-Za-z0-9-_.!() ]+"};
44     return regex_match(model, pattern);
45 }
46
47 // функция валидации поля "фамилия владельца"
48 bool validate_surname(std::string surname) {
49     std::regex pattern{"[ФфРрТтУуХхЦцЧчШшЩщЪъЬьЭэЮюЁёЫыА-Яа-яА-Za-z ]+"};
50     return regex_match(surname, pattern);
51 }
52
53 // функция валидации поля "адрес владельца"
54 bool validate_address(std::string address) {
55     std::regex address_pattern{"[ФфРрТтУуХхЦцЧчШшЩщЪъЬьЭэЮюЁёЫыА-Яа-яА-Za-z0-9!_. ]+"};
56     return regex_match(address, address_pattern);
57 }
58
59 // функция валидации поля "год выпуска"
60 // Здесь проверка не содержит регулярных выражений
61 // Проверка заключается в попытке перевести пользовательский ввод в число,
62 // а затем в проверке подходит ли число временному диапазону
63 bool validate_release_year(std::string release_year) {
64     bool result=true;

```

```

65     if (release_year.size()==0) {
66         printf("Введена пустая строка.\n");
67         return false;
68     }
69     for (int i=0; i<release_year.size(); ++i) {
70         if (!isdigit(release_year[i])) {
71             printf("Строка не полностью состоит из цифр.\n");
72             return false;
73         }
74     }
75     int year=std::stoi(release_year);
76     std::time_t t = std::time(nullptr);
77     std::tm *const pTInfo = std::localtime(&t);
78     int current_year = 1900 + pTInfo->tm_year;
79     if (!(year >= LOWER_BOUND_YEAR && year <= current_year) || year <= 0) {
80         printf("Год вне диапазона(%d-%d)\n", LOWER_BOUND_YEAR, current_year);
81         result = false;
82     }
83     return result;
84 }
85
86 // функция для определения имени и режима открытия файла
87 // Она спрашивает у пользователя имя файла и, если
88 // ОС позволяет использовать данное имя файла, то
89 // файл будет с заданным пользователем именем файла,
90 // иначе надо будет вводить имя файла, пока оно не удовлетворит критериям ОС.
91 std::pair<std::string, std::string> file_info(std::string default_file_name) {
92     std::pair<std::string, std::string> file_info;
93     std::string file_name;
94     printf("Введите название файла(enter=%s): ", default_file_name.c_str());
95     while (scanf("%255[^\n]s", BUFFER)) {
96         clear_stdin();
97         file_name = BUFFER;
98         if (validate_filename(file_name)) {
99             break;
100         }
101         else {
102             printf("Введенное имя файла некорректно. Файл должен иметь вид file.txt,\n");
103             printf("где file состоит из латинских символов, дефиса(-) или нижнего\n");
104             printf("подчеркивания(_),\n");
105             printf("а после следует его расширение .txt\n");
106             printf("Введите название файла(enter=%s): ", default_file_name.c_str());
107         }
108     }
109     if (file_name.size()==0) {
110         file_name = default_file_name;
111     }
112     file_info.first = file_name;
113     // Далее идет проверка существует ли данный файл, если да, то необходимо спросить
114     // у пользователя,
115     // как именно предстоит работать с ним.
116     if (std::filesystem::exists(file_name)) {
117         printf("Файл %s уже существует.\nЖелаете продолжить запись вместо заполнения с\n");
118         printf("начала(enter=Да/Н=Нет)?\n", file_name.c_str());
119         while ((BUFFER[0]=getchar()) != EOF) {
120             clear_stdin();
121             if (BUFFER[0] == '\n') {
122                 file_info.second = "a";
123                 break;
124             } else
125                 if (BUFFER[0] == 'n' || BUFFER[0] == 'N') {

```

```

123         file_info.second = "w";
124         break;
125     } else printf("Введено '%c' можно ввести только enter или N\n", BUFFER[0]);
126 }
127 } else file_info.second = "w";
128 return file_info;
129 }
130
131 // функция записи автомобильных сведений в файл
132 void write_car_data() {
133     std::pair<std::string, std::string> file_information =
134     file_info(DEFAULT_FILENAME_1);
135     std::string file_name = file_information.first;
136     FILE* file = std::fopen(file_name.c_str(), file_information.second.c_str());
137
138     // проверка на открытие файла
139     if (!file) {
140         printf("Не удалось открыть файл %s\n", file_name.c_str());
141         return;
142     }
143     printf("Файл %s успешно открыт для записи\n", file_name.c_str());
144
145     int field_index=0;
146     CarData temp = {};
147     printf("Начинаем заполнять автомобильные сведения.\nДля завершения ввода нажмите
148     'Q' либо 'q'.\n");
149     std::string field_names[3] = {"бренд (символы: A-ЯЁа-яёA-Za-z0-9-_.! )", "модель
150     (символы: A-ЯЁа-яёA-Za-z0-9-_.!() )", "номерной знак (пример: M976MM777RUS)"};
151     printf("Введите поле %s:\n", field_names[field_index].c_str());
152     bool filled = false;
153     int note_number = 0;
154
155     // Ниже идет последовательная запись полей и соответствующих им записей в файл
156     // Если поле не проходит валидацию, то оно не будет занесено в файл,
157     // пока не пройдет валидацию.
158     while (scanf("%255[^\n]s", BUFFER) != EOF) {
159         if ((BUFFER[0]=='Q' || BUFFER[0] == 'q') && BUFFER[1] == '\0') {
160             printf("Введен %s. Ввод сведений закончен\n", BUFFER);
161             clear_stdin();
162             break;
163         }
164         clear_stdin();
165         switch (field_index) {
166             case 0: {
167                 temp.brand.assign(BUFFER);
168                 if (validate_brand(temp.brand))
169                     field_index=(field_index+1)%3;
170                 else printf("Поле введено некорректно, попробуйте еще раз\n");
171                 break;
172             }
173             case 1: {
174                 temp.model.assign(BUFFER);
175                 if (validate_model(temp.model))
176                     field_index=(field_index+1)%3;
177                 else printf("Поле введено некорректно, попробуйте еще раз\n");
178                 break;
179             }
180             case 2: {
181                 temp.license.assign(BUFFER);
182                 if (validate_license(temp.license)) {
183                     field_index = (field_index+1)%3;
184                     filled = true;
185                 }
186             }
187         }
188     }
189 }

```

```

182         } else printf("Поле введено некорректно, попробуйте еще раз\n");
183         break;
184     }
185 }
186 if (filled) {
187     fprintf(file, "%s %s %s\n", temp.brand.c_str(), temp.model.c_str(),
temp.license.c_str());
188     printf(
189         "Запись #%d завершена:\nбренд: %s\nмодель: %s\nномерной знак: %s\n",
190         ++note_number,
191         temp.brand.c_str(),
192         temp.model.c_str(),
193         temp.license.c_str()
194     );
195     printf("Продолжаем запись сведений\n");
196     temp = {};
197     filled = false;
198 }
199 printf("Введите поле %s:\n", field_names[field_index].c_str());
200 }
201 fclose(file);
202 // Здесь происходит очистка потока ввода от EOF
203 clearerr(stdin);
204 return;
205 }
206
207 // функция записи регистрационных сведений в файл
208 // она полностью аналогична функции write_car_data(),
209 // лишь за тем исключением что записывает другие поля
210 void write_license_data() {
211     std::pair<std::string, std::string> file_information =
file_info(DEFAULT_FILENAME_2);
212     std::string file_name = file_information.first;
213     FILE* file = std::fopen(file_name.c_str(), file_information.second.c_str());
214
215     if (!file) {
216         printf("Не удалось открыть файл %s\n", file_name.c_str());
217         return;
218     }
219     printf("Файл %s успешно открыт для записи\n", file_name.c_str());
220
221     int field_index=0;
222     LicenseData temp = {};
223
224     printf("Начинаем заполнять регистрационные сведения.\nДля завершения ввода нажмите
'Q' либо 'q'.\n");
225     std::string field_names[4] = {"номерной знак (пример: M976MM777RUS)", "фамилия
владельца (символы: А-ЯЁа-яёА-Za-z)", "адрес владельца (символы: А-Za-zА-ЯЁа-яё0-9!
_-. )", "год выпуска (символы: 0-9)"};
226     printf("Введите поле %s:\n", field_names[field_index].c_str());
227     bool filled = false;
228     int note_number = 0;
229     while (scanf("%255[^\n]s", BUFFER) != EOF) {
230         if ((BUFFER[0]=='Q' || BUFFER[0] == 'q') && BUFFER[1] == '\0') {
231             printf("Введен %s. Ввод сведений закончен\n", BUFFER);
232             clear_stdin();
233             break;
234         }
235         clear_stdin();
236         switch (field_index) {
237             case 0: {
238                 temp.license.assign(BUFFER);

```

```

239         if (validate_license(temp.license)) {
240             field_index = (field_index+1)%4;
241         } else printf("Поле введено некорректно, попробуйте еще раз\n");
242         break;
243     }
244     case 1: {
245         temp.surname.assign(BUFFER);
246         if (validate_surname(temp.surname)) {
247             field_index = (field_index+1)%4;
248         } else printf("Поле введено некорректно, попробуйте еще раз\n");
249         break;
250     }
251     case 2: {
252         temp.address.assign(BUFFER);
253         if (validate_address(temp.address)) {
254             field_index = (field_index+1)%4;
255         } else printf("Поле введено некорректно, попробуйте еще раз\n");
256         break;
257     }
258     case 3: {
259         temp.release_year.assign(BUFFER);
260         if (validate_release_year(temp.release_year)) {
261             field_index = (field_index+1)%4;
262             filled = true;
263         } else printf("Поле введено некорректно, попробуйте еще раз\n");
264         break;
265     }
266 }
267 if (filled) {
268     fprintf(file, "%s %s %s %s\n", temp.license.c_str(), temp.surname.c_str(),
269     temp.address.c_str(), temp.release_year.c_str());
270     printf(
271         "Запись #%d завершена:\nномерной знак: %s\nфамилия владельца:
272     %s\nадрес: %s\nгод выпуска: %s\n",
273         ++note_number,
274         temp.license.c_str(),
275         temp.surname.c_str(),
276         temp.address.c_str(),
277         temp.release_year.c_str()
278     );
279     printf("Продолжаем запись сведений\n");
280     temp = {};
281     filled = false;
282 }
283 printf("Введите поле %s:\n", field_names[field_index].c_str());
284 }
285 fclose(file);
286 // clear stdin to ignore EOF
287 clearerr(stdin);
288 return;
289 }

```

6

Результаты тестирования

Для демонстрации работы программы заполним файл car_data.txt и выведем его содержимое на экран:

```
→ ./file_h
Назначение программы: Создание файлов на основе сведений, вводимых пользователем
Что требуется записать?
1) Файл автомобильных сведений
2) Файл регистрационных сведений
q) Для завершения программы нажмите q.
1
Введите название файла(enter=car_data.txt):
Файл car_data.txt уже существует.
Желаете продолжить запись вместо заполнения с начала(enter=Да/N=Нет)?

Файл car_data.txt успешно открыт для записи
Начинаем заполнять автомобильные сведения.
Для завершения ввода нажмите 'Q' либо 'q'.
Введите поле бренд (символы: А-ЯЁа-яёА-Za-z0-9-_.! ) :
Dodge
Введите поле модель (символы: А-ЯЁа-яёА-Za-z0-9-_.!() ) :
Charger 1969
Введите поле номерной знак (пример: M976MM777RUS):
M977MM777RUS
Запись #1 завершена:
бренд: Dodge
модель: Charger 1969
номерной знак: M977MM777RUS
Продолжаем запись сведений
Введите поле бренд (символы: А-ЯЁа-яёА-Za-z0-9-_.! ) :
Toyota
Введите поле модель (символы: А-ЯЁа-яёА-Za-z0-9-_.!() ) :
Supra
Введите поле номерной знак (пример: M976MM777RUS):
M976MM766RUS
Запись #2 завершена:
бренд: Toyota
модель: Supra
номерной знак: M976MM766RUS
```

Рис. 1. Пример заполнения первых 2-х записей.

Введем еще одну запись и завершим ввод сведений в файл.

```
Продолжаем запись сведений
Введите поле бренд (символы: А-ЯЁа-яёА-Za-z0-9-_.! ) :
Мерседес
Введите поле модель (символы: А-ЯЁа-яёА-Za-z0-9-_.!() ) :
Модель S
Введите поле номерной знак (пример: М976ММ777RUS):
М976ММ777RUS
Запись #3 завершена:
бренд: Мерседес
модель: Модель S
номерной знак: М976ММ777RUS
Продолжаем запись сведений
Введите поле бренд (символы: А-ЯЁа-яёА-Za-z0-9-_.! ) :
q
Введен q. Ввод сведений закончен
```

Рис. 2. Пример заполнения еще одной записи.

Опять видим основные опции программы. Завершим работу с программой.

```
Что требуется записать?
1) Файл автомобильных сведений
2) Файл регистрационных сведений
q) Для завершения программы нажмите q.
q
Нажмите любую клавишу для завершения программы...
Программа завершена.
```

Рис. 3. Снова основное меню.

Выведем на экран содержимое файла car_data.txt.

```
→ cat car_data.txt
Dodge Charger 1969 М977ММ777RUS
Toyota Supra М976ММ766RUS
Мерседес Модель S М976ММ777RUS
```

Рис. 4. Содержимое car_data.txt.

Можно заметить, что информация записалась в файл без каких либо проблем.

Теперь запишем информацию в файл license_data.txt:

```
→ ./file_h
Назначение программы: Создание файлов на основе сведений, вводимых пользователем
Что требуется записать?
1) Файл автомобильных сведений
2) Файл регистрационных сведений
q) Для завершения программы нажмите q.
2
Введите название файла(enter=license_data.txt): license_data.txt
Файл license_data.txt успешно открыт для записи
Начинаем заполнять регистрационные сведения.
Для завершения ввода нажмите 'Q' либо 'q'.
Введите поле номерной знак (пример: M976MM777RUS):
M977MM777RUS
Введите поле фамилия владельца (символы: A-ЯЁа-яёA-Za-z):
Pushkin
Введите поле адрес владельца (символы: A-Za-zA-ЯЁа-яё0-9!_-. ):
Ул. Новые Черемушки
Введите поле год выпуска (символы: 0-9):
2008
Запись #1 завершена:
номерной знак: M977MM777RUS
фамилия владельца: Pushkin
адрес: Ул. Новые Черемушки
год выпуска: 2008
Продолжаем запись сведений
Введите поле номерной знак (пример: M976MM777RUS):
q
Введен q. Ввод сведений закончен
Желаете продолжить запись? (enter=Да/N=Нет)
n
Нажмите любую клавишу для завершения программы...
Программа завершена.
```

Рис. 5. Пример заполнения license_data.txt.

Теперь выведем содержимое файла на экран.

```
→ cat license_data.txt
M977MM777RUS Pushkin Ул. Новые Черемушки 2008
```

Рис. 6. Содержимое license_data.txt.

Теперь допишем информацию в license_data.txt.

```
→ ./file_h
Назначение программы: Создание файлов на основе сведений, вводимых пользователем
Что требуется записать?
1) Файл автомобильных сведений
2) Файл регистрационных сведений
q) Для завершения программы нажмите q.
2
Введите название файла(enter=license_data.txt):
Файл license_data.txt уже существует.
Желаете продолжить запись вместо заполнения с начала(enter=Да/N=Нет)?

Файл license_data.txt успешно открыт для записи
Начинаем заполнять регистрационные сведения.
Для завершения ввода нажмите 'Q' либо 'q'.
Введите поле номерной знак (пример: M976MM777RUS):
M976CC777RUS
Введите поле фамилия владельца (символы: А-ЯЁа-яёА-Zа-z):
&
Поле введено некорректно, попробуйте еще раз
Введите поле фамилия владельца (символы: А-ЯЁа-яёА-Zа-z):
Nikitin
Введите поле адрес владельца (символы: А-Zа-zА-ЯЁа-яё0-9!_-. ):
Улица Профсоюзная д.26
Введите поле год выпуска (символы: 0-9):
1977
Запись #1 завершена:
номерной знак: M976CC777RUS
фамилия владельца: Nikitin
адрес: Улица Профсоюзная д.26
год выпуска: 1977
Продолжаем запись сведений
Введите поле номерной знак (пример: M976MM777RUS):
q
Введен q. Ввод сведений закончен
Желаете продолжить запись? (enter=Да/N=Нет)
n
Нажмите любую клавишу для завершения программы...
Программа завершена.
```

Рис. 7. Пример дозаписи сведений в license_data.txt.

Стоит отметить, что при заполнении поля фамилия была совершена ошибка. Программа заметила, что поле введено некорректно, и потребовала перезаписать поле.

Теперь посмотрим содержимое license_data.txt

```
→ cat license_data.txt
M977MM777RUS Pushkin Ул. Новые Черемушки 2008
M976CC777RUS Nikitin Улица Профсоюзная д.26 1977
```

Рис. 8. Содержимое license_data.txt после дозаписи.

В результате небольшого тестирования можно отметить, что программа функционирует корректно.